DESIGN DOC V0.1

# Jarvis Daily Briefing — Architecture

A self-contained daily-briefing artifact: a polished HTML page with an audio-reactive 3D orb that reads you a Jarvis-style spoken summary of time, weather, top India news, and your sprint board for the day. This document explains the design decisions and the trade-offs that shaped them.

Repository: `github.com/archowdhury16/jarvis-daily-briefing` · Author: Arindam Chowdhury · Generated 30 April 2026

## 1. The constraint that shaped everything

The naive design — "*a launchd cron job that hits Jira's REST API, generates a briefing, and plays audio*" — is impossible in this environment because of three independent walls:

| Wall | Where it bit | Symptom |
|---|---|---|
| **Q2's gateway rejects Basic Auth** | Atlassian REST `/myself` → 401 | `WWW-Authenticate: OAuth realm` header — Basic Auth disabled corp-wide |
| **Personal API tokens have no Jira scope** | Token creation page | Only `read:me` / `read:account` scopes available |
| **TLS-inspecting proxy re-signs every cert** | httpx, Node, urllib3 | `SELF_SIGNED_CERT_IN_CHAIN` against every API |

The only thing that *can* read Jira is **Claude TUI** (it has an OAuth-authenticated `claude.ai Atlassian` MCP server). Headless `claude -p` doesn't inherit those MCPs. Cowork *cloud* scheduled tasks have MCP but run in a sandbox with no access to the Mac's speakers or filesystem.

That's why no single binary can do the whole job. The architecture is a **deliberate split** between fetch and render.

## 2. The pipeline

```
                    ┌──────────────────────────┐
                    │  USER TRIGGER (3 surfaces)  │
                    └──────────────────────────┘
                                  │
                                  │
     ┌────────────────────────────┼────────────────────────────┐
     │                            │                            │
/briefing                  Daily Briefing.command       briefing (last).webloc
(Claude TUI)               (desktop double-click)       (Chrome bookmark)
     │                            │                            │
fresh tickets                stale tickets                zero work —
via MCP                       (last fetched)            opens cached HTML
     │                            │                            │
     └──────────────┬─────────────┘                            │
                    ▼                                          │
                                                               │
          ┌──────────────┐                                     │
          │  briefing.py  │ ← single Python entry point        │
          │ (the engine)  │                                     │
          └──────────────┘                                     │
                  │                                            │
                  │                                            │
     ┌────────────┼────────────┬────────────┬────────────┐     │
     ▼            ▼            ▼            ▼            ▼     │
 weather      news RSS      tickets      prose        TTS      │
open-meteo   Google News   JSON file    template     EL → KK   │
                           (or REST)    (no LLM)               │
     │            │            │            │            │     │
     └────────────┴────────────┼────────────┴────────────┘     │
                               ▼                               │
                                                               │
                       ┌──────────────┐                        │
                       │ briefing.html │ ← single 2.5MB file ┘
                       (audio embedded
                        as base64)
                       └──────────────┘
                               ▼
                           [browser]
                               │
                               ▼
                       ┌──────────────┐
                       │ Three.js orb +
                       Web Audio API
                       amp → shader
                       uniform on click
                       └──────────────┘
```

## 3. The trigger surface — three entry points, one engine

The same `briefing.py` is invoked by all three triggers; what differs is **how the tickets get there**.

- **`/briefing` (Claude TUI)** — the only path that gets fresh Jira data. The slash command is a markdown file at `~/.claude/commands/briefing.md` that tells Claude to call its Atlassian MCP, transform the response into our schema, write `tickets.json`, then shell out to `briefing.py`. Claude is the OAuth-authenticated reader here; the script just consumes the JSON.
- **`Daily Briefing.command`** (desktop) — a zsh wrapper that runs `briefing.py --tickets tickets.json`. Reuses whatever was fetched last, regenerates everything else. Useful for midday refreshes or when Claude is closed.
- **`Daily Briefing (last).webloc`** — pure replay. Opens the cached HTML directly in Chrome. Zero regeneration.

The `briefing.py` engine is the same code path for all three — it doesn't care who fed it tickets.

---

## 4. The data fetch layer

Four independent fetchers with graceful degradation:

- **Weather** — `open-meteo.com/v1/forecast`, free, no key, no auth. Returns current temp, apparent, daily H/L. If it fails, the script aborts (cheap, fast, almost never down).
- **News** — Google News RSS (no key). Parsed via `xml.etree.ElementTree`. Filters out exam-result spam common in the IN feed. Strips trailing `" - Source"` suffix Google appends.
- **Tickets** — three modes selected at runtime: `--tickets path.json` (works behind any firewall), `[jira]` block + Basic-Auth REST (works for non-Q2 orgs), or `--no-tickets`.
- **Time** — `datetime.now(ZoneInfo(cfg.timezone))` — no fetch, just zone math.

Each function takes `cfg` and a `verify` arg (truststore / CA bundle / `True`) and returns a dict or list. They're independent — adding a fifth source (calendar, oncall, etc.) is one new function plus a sentence in `build_prose()`.

---

## 5. Prose generation — deterministic, no LLM

A deliberate choice, not a limitation. `build_prose(cfg, weather, news, tickets, now)` is a pure function that assembles the briefing from f-string templates. Why no LLM:

1. **Reproducibility** — same inputs, identical prose. No "today George got chatty" variance.
2. **Latency** — runs in <1ms. An LLM call would add 2–5s and a network dependency.
3. **No leakage** — ticket summaries never leave your laptop until the TTS step (and even there, only the final prose, not raw JSON).
4. **Determinism for TTS chunking** — same prose every time means audio caching could work later if needed.

The template handles three structural decisions: greeting word from hour-of-day; priority-tier sort (Highest → P4 → None) with overdue items floated to the top; ticket-key spelling-out ( `GD-11065` → "*G D dash one one zero six five*") so TTS pronounces them as identifiers, not as one mangled word.

---

## 6. TTS layer — primary + fallback chain

```
synthesize_audio(text)
    │
    ├─► try_elevenlabs(text)          [primary: George, ~1.3K chars/run]
    │       └── HTTPS POST → eleven_turbo_v2_5
    │           returns bytes (mp3) → return (bytes, "audio/mpeg")
    │       ✗ on quota / network / missing-key → fall through
    │
    └─► try_kokoro(text)              [fallback: am_adam, fully local]
            └── KPipeline → numpy float32 frames → soundfile WAV
                returns bytes (wav) → return (bytes, "audio/wav")
            ✗ on import failure → return None (silent artifact, with warning)
```

Two-tier fallback because: ElevenLabs is best-quality but quota-limited (10K chars free, ~7 daily briefings); Kokoro keeps you running indefinitely on free tier; both share the same calling contract — bytes + MIME type — so the artifact builder doesn't care which won. The MIME type matters: the embedded `<audio>` tag uses `data:${mime};base64,${b64}`. Browsers won't play wav as mpeg or vice versa.

---

## 7. The artifact — the clever bit

`build_artifact()` reads `templates/briefing.html`, replaces `__INJECT_DATA__` with `json.dumps(data)`, writes the result. The trick is **what's in `data`**:

```
{
  "greeting": "Good morning",
  "weather": {…},
  "news": [{…},…],
  "tickets": [{…},…],
  "audio_b64": "<1.7MB of base64>",
  "audio_mime": "audio/mpeg",
  …
}
```

The audio is **embedded as base64** directly into the HTML. The artifact is a single 2.5 MB file with **zero external network calls when you open it**. It works offline, behind any firewall, on a plane. You can email it. You can open it on a phone. The orb shader resources are loaded from CDN once and cached forever (Three.js + Inter font); they're not gated.

This is why the same file works as `/briefing` output, `Daily Briefing.command` output, and `(last).webloc` replay target — it's literally the same self-contained file.

## 8. Audio-reactive orb — the rendering pipeline

```
click orb
   │
   ▼
audio.play()
   │
   ▼
Web Audio API: createMediaElementSource → AnalyserNode (fftSize=128, smoothing=0.8)
   │
   ▼ (every frame, ~60 FPS via requestAnimationFrame)
   │
analyser.getByteFrequencyData(dataArr)  → 64 frequency bins
   │
   ▼
avg of bins[2..29]  (low-mid, where speech energy lives)
   │
   ▼
exponential smoothing: amp += (target - amp) * dt * 8
   │
   ▼
uniforms.uAmp.value = amp     ← passed to GPU
   │
   ▼
┌──────────── vertex shader ────────────┐     ┌──── fragment shader ────┐
│ disp = 0.18·snoise(p·1.4)             │     │ fresnel = (1-N·V)²      │
│      + 0.10·snoise(p·2.7)             │     │ col = mix(A,B) iridescent│
│      + uAmp·0.45·snoise(p·4.0)        │     │ col += highlight + rim  │
│ pos = pos + normal * disp             │     │ col *= 0.72  (we tuned  │
└───────────────────────────────────────┘     │   brightness here)      │
                                              └─────────────────────────┘
```

Two key design choices in the shader:

- **Brightness is not audio-reactive** (anymore). Color multiply is constant `0.72`. Only motion (rotation speed, scale ~5%, surface displacement) reacts. This was the "*too bright while speaking*" fix.
- **Simplex noise on three frequencies** — gives the orb organic, non-repeating motion that doesn't look like a tessellation. The three octaves (1.4, 2.7, 4.0) layer detail; the third is the one that scales with `uAmp` so loud syllables make finer ripples.

## 9. What we tried that didn't work

This shaped the final design more than the requirements did:

- **OpenJarvis local stack** (Ollama + Kokoro + custom Jira connector). Built it. Works for everything except *getting Jira data* — Q2 blocks Basic Auth at the gateway. Sits unused on disk now.
- **Cowork cloud scheduled task**. Has MCP, runs on schedule. But sandboxed: cannot reach Mac speakers, cannot reliably write to user filesystem, ignored prompt updates due to caching. Disabled.
- **Headless `claude -p` from launchd**. Has speakers + filesystem. But doesn't inherit `claude.ai`-managed MCPs. Could work with manually-registered MCPs, but those need their own OAuth flow per machine.
- **Atlassian REST + corporate CA bundle**. Auth works (token recognized) but Q2 only allows `read:me` / `read:account` scopes for personal tokens, so all Jira queries return zero results.
- **GLB 3D model in the orb slot**. Reverted — the procedural shader sphere read better.

The final architecture survives because it doesn't try to bridge worlds that can't be bridged. Claude TUI does the *one thing only it can do* (read Jira via authenticated MCP); everything else runs locally.

---

## 10. Limitations & future moves

- `/briefing` **requires Claude TUI to be running** at trigger time. No truly autonomous 8 AM weekday firing. The launchd plist exists but is intentionally not loaded — without working auth, it'd produce empty tickets.
- **ElevenLabs free tier is ~7 days** of briefings. The Kokoro fallback is silent-graceful but quality drops. $5/mo is the painless answer when you're tired of the drop.
- **Headlines are India-only by default**. Easy to flip via `[news]` config ( `rss_url` ).
- **No cache invalidation for the audio**. If you `--no-tts` then later regenerate, you get fresh audio. But there's no "synthesize once per prose hash" optimization — would save ~1.3K chars per re-run if you swap weather mid-day. ~5 lines of code if it becomes load-bearing.
- **Single user runtime**. The repo is now portable to any user (after the `keychain_account = "auto"` change), but the runtime is single-tenant — one config.toml, one tickets.json. Multi-user would mean parametrizing paths or adding a `--profile` flag.

> If you wanted to go further: a web-app version with server-side state and per-user OAuth would unlock truly autonomous schedules. But the local-only, self-contained design is what made this *quick* to ship and zero-cost to run.

---